

Learning Push Policies via Surrogate Combinatorial Optimization and Differentiable Physics

Jeremy Morgan

Changhui Hou

Leland Ling

University of Southern California

University of Southern California

University of Southern California

May 2026

Abstract

We study the problem of learning effective push policies for planar robotic manipulation without demonstrations. Given a T-shaped block and a target pose, a circular pusher must select a face to contact, a contact point along that face, and a push angle so as to bring the block as close as possible to the goal. This discrete-continuous action space poses a fundamental challenge for gradient-based learning: the face-selection step is combinatorial and breaks the differentiable path from actions to outcomes.

We address this by adapting the *SurCo* framework [Ferber et al., 2023], which trains a neural network to predict the cost parameters of a Mixed-Integer Quadratic Program (MIQP) whose solution is the push action. Gradients are propagated back through the Gurobi solver using randomized smoothing [Berthet et al., 2020], while the physics rollout is handled by a fully differentiable JAX-based simulator [Ferigo et al., 2023]. We evaluate our system on four different experiments and comment on future directions for the work. Our code is available at <https://github.com/jstmn/PushT-with-Optimization>.

1 Introduction

Robotic manipulation of ungraspable objects through pushing is a foundational problem in robotics [Mason, 1986, Lynch and Mason, 1992]. Even a single planar push requires reasoning about geometry (which face to contact), kinematics (where along the face to push), and dynamics (at what angle to apply force). Because the face-selection step is inherently discrete, classical differentiable-policy methods cannot directly optimize over the full action space.

Central research question. *Can a neural network, trained purely via simulated rollouts and without expert demonstrations, learn to predict MIQP solver parameters that produce effective push actions—and can this be done using gradients propagated through the combinatorial solver?*

The PushT environment [Chi et al., 2023] instantiates this challenge concretely: a circular pusher must manipulate a rigid T-shaped block (20 cm $\times$ 25 cm) inside a 1.5 m $\times$ 1.5 m planar workspace to a randomly sampled goal pose. The environment is simulated in JAXsim [Ferigo et al., 2023], enabling fully differentiable physics rollouts.

Contributions.

1. We formulate the PushT manipulation problem as a contextual decision-focused learning problem, where a neural network predicts the surrogate cost of an MIQP whose solution is the executed action.
2. We implement a SurCo-style pipeline, connecting a learned cost predictor with Gurobi [Gurobi Optimization, LLC, 2023] via a custom JAX VJP that uses randomized smoothing [Berthet et al., 2020].
3. We perform 4 experiments as well as in-depth analysis on the gradients produced by smoothing gradients to identify potential issues with the approach.

2 Related Work

Our work sits at the intersection of five lines of research: learning-augmented combinatorial optimization (with SurCo as the most direct precursor), gradient estimation through discrete solvers, differentiable physics for control, planar pushing as a manipulation benchmark, and mixed-integer programming for contact-rich manipulation.

Predict-then-optimize and SurCo. The predict-then-optimize paradigm trains a model whose output is consumed by a downstream solver, with loss measured on decision quality rather than prediction accuracy [Elmachtoub and Grigas, 2022]. Amos and Kolter [2017] showed how to embed a quadratic program as a differentiable layer via implicit differentiation of its KKT conditions, but this requires a convex, continuous solver. SurCo [Ferber et al., 2023] extends the framework to combinatorial solvers by training a network to predict a *linear surrogate cost vector* that, when passed to an off-the-shelf MILP solver, yields a low true-cost decision.

Differentiating through combinatorial solvers. Because the arg min of an integer program is piecewise constant, standard backpropagation yields zero gradients almost everywhere. Pogančić et al. [2020] address this by constructing a surrogate gradient via interpolation between solver outputs at the current and a perturbed cost. The perturbed optimizers of Berthet et al. [2020], which we adopt, instead average solutions under Gaussian perturbations of the cost vector, defining a smoothed objective whose gradient admits an unbiased Monte Carlo estimator.

Differentiable physics for manipulation. Differentiable simulators expose analytic gradients of trajectories with respect to actions, enabling first-order policy optimization through contact-rich dynamics. de Avila Belbute-Peres et al. [2018] introduced a differentiable rigid-body engine via a linear complementarity formulation of contact. We use JAXsim [Ferigo et al., 2023], a fully differentiable multi-body dynamics engine in JAX [Bradbury et al., 2018], to roll out the pusher-block interaction and obtain gradients of the task loss with respect to continuous action components. The discrete face-selection step breaks this differentiable path; randomized smoothing bridges the gap.

Planar pushing and the PushT benchmark. Pushing as a non-prehensile manipulation primitive has been studied since Mason [1986] and Lynch and Mason [1992]. The PushT environment was popularized as a benchmark for imitation learning [Chi et al., 2023, Florence et al., 2022], where policies are trained from human demonstrations. Our approach is demonstration-free—the training signal comes entirely from simulator rollouts—and the action is produced by a combinatorial solver whose cost parameters, rather than the action itself, are emitted by the network.

3 Problem Formulation

We now formally define the decision problem.

3.1 Environment and State

The environment consists of a T-shaped block with pose $q_T = (p_T, \theta_T) \in \mathbb{R}^2 \times [0, 2\pi)$ and a fixed goal pose $q_{\text{goal}} = (p_{\text{goal}}, \theta_{\text{goal}}) \in \mathbb{R}^2 \times [0, 2\pi)$. The pusher is a rigid disk whose trajectory is determined entirely by the push action.

Context observed at decision time. Before each push, the agent observes a 5-dimensional *context vector* expressed in the target’s body frame:

$$y = (\Delta x^{(T)}, \Delta y^{(T)}, \Delta \theta, \dot{x}^{(T)}, \dot{y}^{(T)}).$$

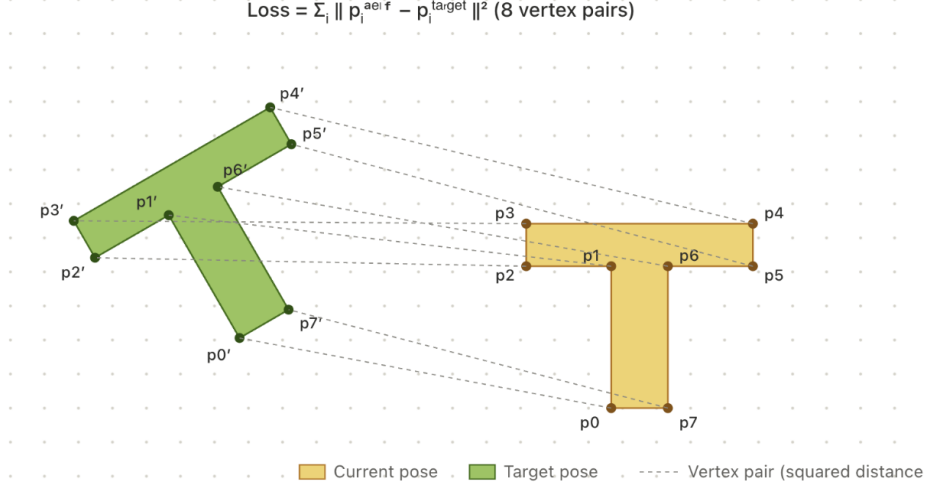


Figure 1: The T and target T pose visualized. The objective function is the square of the distances between each of the corner points.

The first two entries are the position of the T-block relative to the target, rotated into the target’s frame:

$$\begin{aligned}\Delta x^{(T)} &= (x_T - x_{\text{goal}}) \cos \theta_{\text{goal}} + (y_T - y_{\text{goal}}) \sin \theta_{\text{goal}}, \\ \Delta y^{(T)} &= -(x_T - x_{\text{goal}}) \sin \theta_{\text{goal}} + (y_T - y_{\text{goal}}) \cos \theta_{\text{goal}}.\end{aligned}$$

The third entry $\Delta \theta = \theta_T - \theta_{\text{goal}}$ is the orientation error, and $(\dot{x}^{(T)}, \dot{y}^{(T)})$ is the T-block’s linear velocity, also rotated into the target frame. No future-state information is available; the action must be chosen from the current configuration alone.

By design, this representation is invariant to global translations and rotations of the (T, target) pair: any rigid transformation of the workspace that preserves the relative pose leaves y unchanged. This bakes the task’s SE(2) symmetry directly into the input rather than requiring the network to learn it from data.

3.2 Decision Variables

A single push action is parameterized by three components:

- $f \in \{0, 1, 2, 3\}$: the face of the T-block to contact (4 faces after geometric restriction). We represent this as a one-hot vector $\mathbf{f} \in \{0, 1\}^{|\mathcal{F}|}$ with $|\mathcal{F}| = 4$ in the MIQP.
- $s \in [0.2, 0.8]$: the normalized contact point along the chosen face.
- $\alpha \in [\pi/4, 3\pi/4]$: the push angle (direction of force application).

The full single-step decision variable is $x = (\mathbf{f}, s, \alpha)$. For a multi-step horizon of length M , the decision variable is the sequence $\mathbf{x} = (x_1, \dots, x_M)$.

3.3 Objective

The *task cost* after executing action x from context y is the sum of squared distances between the final block positions and their targets:

$$L(x; y) = \sum_{i=1}^N \|p_i^{\text{final}}(x, y) - p_i^{\text{target}}(y)\|_2^2, \quad (1)$$

where $p_i^{\text{final}}(x, y)$ is the position of block i after executing action x from context y , and $p_i^{\text{target}}(y)$ is its goal position. This formulation avoids the need of minimizing an angular difference, thereby making for an easier learning problem.

The learning objective is to minimize the expected task cost over a distribution of initial configurations:

$$\min_{\theta} \mathbb{E}_{y \sim \mathcal{D}} [L(\pi_{\theta}(y); y)], \quad (2)$$

where π_{θ} is the learned policy parameterized by neural network weights θ , and $\mathcal{D}$ is the distribution over initial T-block and goal poses.

The policy π_{θ} operates by predicting a *cost vector* $c \in \mathbb{R}^{|\mathcal{F}|+2}$ and then solving the following Mixed-Integer Quadratic Program:

$$\begin{aligned} x^* = \arg \min_x \quad & \sum_{i=1}^{|\mathcal{F}|} c_i x_i + \lambda_s (x_{|\mathcal{F}|+1} - c_{|\mathcal{F}|+1})^2 + \lambda_{\alpha} (x_{|\mathcal{F}|+2} - c_{|\mathcal{F}|+2})^2 \\ \text{subject to} \quad & \sum_{i=1}^{|\mathcal{F}|} x_i = 1, \\ & x_i \in \{0, 1\} \quad \text{for } i = 1, \dots, |\mathcal{F}|, \\ & x_{|\mathcal{F}|+1} \in [0.2, 0.8], \\ & x_{|\mathcal{F}|+2} \in [\pi/4, 3\pi/4]. \end{aligned} \quad (3)$$

The cost vector c encodes the network’s preferences over the action: its first $|\mathcal{F}|$ entries score the relative desirability of each face, while the last two entries specify target values for the continuous contact point and angle. The one-hot constraint selects exactly one face, making (3) a true MIQP solved by Gurobi [Gurobi Optimization, LLC, 2023]. Here $|\mathcal{F}| = 4$ is the number of candidate faces. We use a quadratic objective on the continuous variables, rather than a linear one, so that the optimal contact point and angle can take values in the interior of their feasible ranges: a MILP formulation would always place them at a vertex of the feasible box, collapsing the continuous degrees of freedom to their bounds and giving the network only a discrete set of action choices.

What is learned. The neural network $g_{\theta} : \mathbb{R}^5 \rightarrow \mathbb{R}^6$ maps the context y to the cost parameters $(c_{\mathbf{f}}, s^*, \alpha^*)$. The weights θ are learned end-to-end by differentiating the task loss (1) through the MIQP solver and the physics simulator.

4 Approach

Figure 2 gives an overview of the full pipeline.

4.1 Context Encoder (MLP)

The cost predictor is a multi-layer perceptron with a shared trunk and two output heads:

$$\begin{aligned} h_{\theta} : \mathbb{R}^5 & \xrightarrow{\text{Dense}(128)} \xrightarrow{\text{ReLU}} \xrightarrow{\text{Dense}(128)} \xrightarrow{\text{ReLU}} \mathbb{R}^{128}, \\ c_{\mathbf{f}} &= \text{Dense}_{\text{face}}(h_{\theta}(y)) \in \mathbb{R}^{|\mathcal{F}|}, \\ (s^*, \alpha^*) &= \text{Dense}_{\text{cont}}(h_{\theta}(y)) \in \mathbb{R}^2. \end{aligned}$$

The shared trunk h_{θ} maps the context y to a 128-dimensional latent; the face head produces the discrete cost vector $c_{\mathbf{f}}$, and the continuous head produces the targets (s^*, α^*) for the contact point and angle. The continuous head $\text{Dense}_{\text{cont}}$ is initialized so that its outputs map to the centers of the variable ranges, i.e. $s^* = 0.5$ and $\alpha^* = \pi/2$, which lie at the midpoints of $[0.2, 0.8]$ and $[\pi/4, 3\pi/4]$ respectively. This keeps the targets well inside the feasible region at the start of training, so the MIQP solution is not pinned to a box boundary before the network has learned anything meaningful.

Note that the angular difference between the current T and the goal pose is a rotational value. This presents a potential issue whereby the value of 0 and π are equivalent mathematically but numerically different. There exists no explicit handling of this except for a remapping of all rotational values to $[-\pi, \pi]$. One trick used in spatial computing is to pass $[\sin(\theta), \cos(\theta)]$ instead of θ to the network. Alternatively, using only the difference in x and y components of each corner point would consistent a representation free of rotations. While mathematically clever, such tricks have exhibited negligible impacts in previous robotics work of the authors.

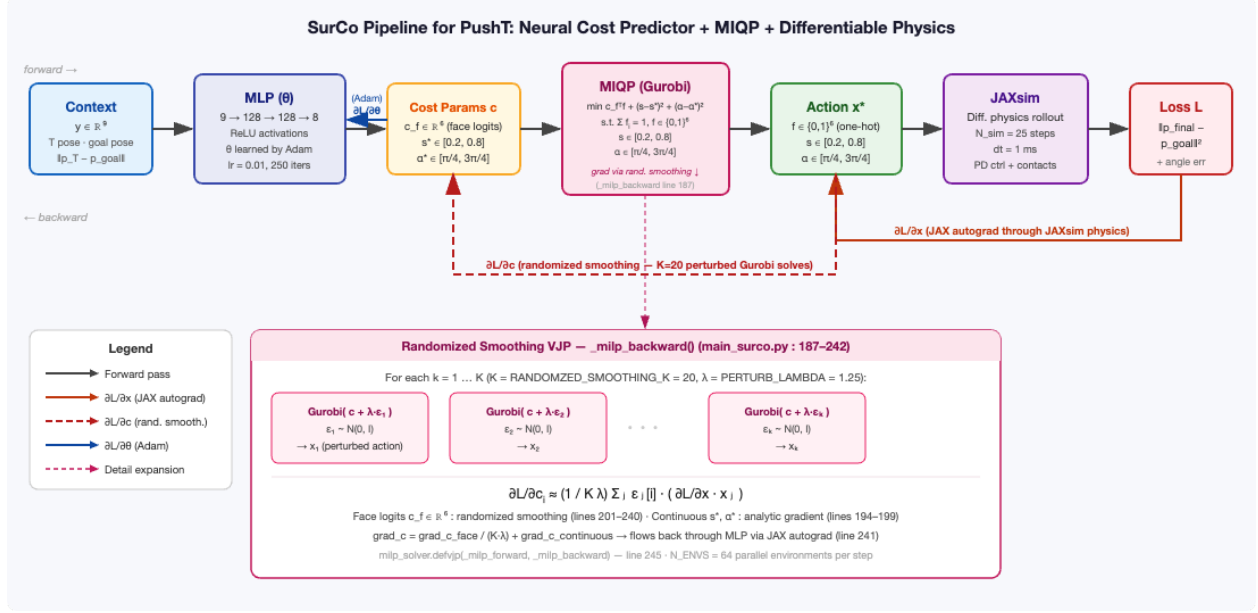


Figure 2: **SurCo pipeline for PushT**. The MLP maps the observed context to MIQP cost parameters. Gurobi solves the MIQP to select an action. The action is executed in JAXsim. Gradients flow back through the physics rollout (solid lines) and through the solver via randomized smoothing (dashed lines).

4.2 MIQP Solver (Gurobi)

We use Gurobi [Gurobi Optimization, LLC, 2023] to solve (3). The binary face variables are modeled as a set of general integer variables with bounds $[0, 1]$ and an equality constraint $\sum_{i=1}^{|\mathcal{F}|} f_i = 1$. The quadratic terms are handled natively by Gurobi’s MIQP solver. For efficiency, we maintain a *persistent* Gurobi model across iterations and update only the objective coefficients.

4.3 Differentiable Physics (JAXsim)

Each push is simulated in JAXsim [Ferigo et al., 2023] with a physics time step of $\Delta t = 1$ ms. At the start of every action, the pusher is teleported to a *start pose* located 5 mm behind the planned contact point, along the push direction, with its velocity reset to zero. From there, a PD controller (gains $K_P = 400$, $K_D = 40$, saturation 80 N) tracks a linearly interpolated target trajectory with two phases:

1. **Approach:** The pusher closes the 5 mm gap to the pre-contact point at 8 m/s.
2. **Push:** The pusher advances 30 cm along the push direction at 1 m/s.

Because JaxSim only supports point-vs-terrain contacts, pusher–block interaction is modeled by an analytical penalty spring-damper ($K_{\text{contact}} = 2 \times 10^4$ N/m, $D_{\text{contact}} = 80$ N·s/m), which activates only while the pusher penetrates the block’s signed distance field and only while the contact is closing. The eight corners of the T are similarly checked against the workspace walls with a spring-damper of the same stiffness and damping. Block–floor interaction is handled in joint space with JaxSim’s terrain friction disabled: it combines a Coulomb term μmg and a viscous term $c_{\text{lin}} v$, both smoothed by $\tanh(v/v_\epsilon)$ for differentiability, with an analogous rotational friction term acting on the yaw degree of freedom.

4.4 Gradient Estimation via Randomized Smoothing

Custom JAX VJP. We register a custom vector–Jacobian product for the solver call. Given an upstream gradient $\bar{L} = \partial L / \partial x^*$, the gradient with respect to the cost vector is decomposed into face and continuous components, each estimated by a different mechanism.

Face logits are perturbed and the resulting actions executed in the physics simulator. With M Monte

Carlo draws, the face gradient is

$$\frac{\partial L}{\partial c_i^{(\mathbf{f})}} \approx \frac{1}{M\lambda} \sum_{k=1}^M \varepsilon_k^{(i)} (L_k - L_0), \quad \varepsilon_k \sim \mathcal{N}(0, I_{|\mathcal{F}|}), \quad (4)$$

where $L_k = L(x^*(c + \lambda \varepsilon_k); y)$ is the task cost of the rollout induced by the perturbed solve and $L_0 = L(x^*(c); y)$ is the task cost of the unperturbed action. Subtracting the unperturbed cost L_0 acts as a mean-baseline control variate, which substantially reduces the variance of the estimator without introducing bias.

Continuous targets (s^*, α^*) are not perturbed. Their gradients are obtained by backpropagating through the physics rollout itself: holding the face one-hot fixed at their perturbed values, we evaluate the rollout cost as a differentiable function of (s^*, α^*) and apply `jax.value_and_grad`. The resulting gradient is averaged across the M perturbed face draws so that the continuous update accounts for the same distribution of face choices used by the face estimator.

We use $M = 9$ Monte Carlo draws and a perturbation scale $\lambda = 1.25$.

4.5 Training Procedure

Algorithm 1 summarizes the training loop. The regularization term penalizes face logits and continuous

Algorithm 1 SurCo Training for PushT

Require: Initial weights θ , learning rate η , iterations T , number of parallel envs N , MC draws M , perturbation scale λ

```

1: for  $t = 1, \dots, T$  do
2:   Sample  $N$  environment configurations  $\{y^{(i)}\}_{i=1}^N$ 
3:    $c^{(i)} \leftarrow g_\theta(y^{(i)})$  ▷ forward pass through MLP
4:    $x^{*(i)} \leftarrow \text{Gurobi}(c^{(i)})$  ▷ solve MIQP per env
5:    $L_0^{(i)} \leftarrow L(\text{rollout}(x^{*(i)}, y^{(i)}))$  ▷ unperturbed rollout in JAXsim
6:   for  $k = 1, \dots, M$  do
7:     Sample  $\varepsilon_k^{(i)} \sim \mathcal{N}(0, I_{|\mathcal{F}|})$  on the face block
8:      $x_k^{(i)} \leftarrow \text{Gurobi}(c^{(i)} + \lambda \varepsilon_k^{(i)})$ 
9:      $L_k^{(i)} \leftarrow L(\text{rollout}(x_k^{(i)}, y^{(i)}))$ 
10:  end for
11:   $\mathcal{L} \leftarrow \frac{1}{N} \sum_i L_0^{(i)} + \mathcal{R}(c^{(i)})$ 
12:  Compute  $\nabla_\theta \mathcal{L}$ : face gradient via (4), continuous gradient via backprop through the rollout, then chain rule through  $g_\theta$ 
13:   $\theta \leftarrow \text{SGD}(\theta, \nabla_\theta \mathcal{L}, \eta)$ 
14: end for
```

targets that stray from the centers of their feasible ranges:

$$\mathcal{R}(c) = \beta_{\mathbf{f}} \overline{\|c_{\mathbf{f}}\|_2^2} + \beta_s \left(\frac{s^* - \bar{s}}{\Delta s} \right)^2 + \beta_\alpha \left(\frac{\alpha^* - \bar{\alpha}}{\Delta \alpha} \right)^2,$$

where $\bar{\cdot}$ denotes a mean over envs (and faces, for the first term), $\bar{s} = 0.5$ and $\bar{\alpha} = \pi/2$ are the midpoints of the feasible ranges, and $\Delta s = 0.6$ and $\Delta \alpha = \pi/2$ are their widths. We use $\beta_{\mathbf{f}} = 0.25$ and $\beta_s = \beta_\alpha = 0.1$. The continuous penalties are normalized by the range widths so the three regularizers have comparable units. Training uses SGD with $\eta = 0.05$, $N = 64$ parallel environments, $M = 9$ Monte Carlo draws, $\lambda = 1.25$, and runs for up to $T = 10000$ iterations. We deliberately use plain SGD rather than Adam because the loss landscape here is piecewise: whenever the perturbed solves flip to a different face, the gradient direction can change abruptly. Adam’s momentum and per-parameter running statistics are accumulated under the assumption that recent gradients are predictive of future ones, which is exactly what fails across these discrete transitions — the buffered momentum becomes stale and pulls the optimizer in a direction that no longer reflects the current face configuration. SGD has no such memory, so it adapts to the new gradient direction immediately.

5 Experiments

While continuous control is the long-term goal of this project, we first test whether the system can learn to make effective *single pushes*. For all experiments, we use SGD with a learning rate of 0.05.

We do not report results for closed-loop continuous control. The infrastructure for evaluating a trained policy over continuous rollouts was built, but since the trained policies performed no better than random on T-blocks with randomized poses, the metric we care about — the average number of pushes to reach the target — would not be meaningful.

Metric. We report the *distance change* over the rollout, defined as the difference between the task loss at the final and initial timesteps:

$$\Delta L = L(x; y)|_{t=T} - L(x; y)|_{t=0},$$

where L is the sum of squared distances between block and target corner positions from (1). Lower is better: a negative ΔL means the pusher has moved the block closer to its goal configuration. For each rollout $T=25$ simulation steps are taken.

Experiment 1: 1-env-fixed.

A single environment with both the target pose and the initial T-block pose held fixed across every training iteration. This is the easiest possible setting: the network sees the same context y at every step, so the task reduces to finding a single action that minimizes ΔL for one configuration. It serves as a sanity check that the custom VJP and the gradient pipeline are working — if the method cannot drive ΔL down here, nothing more ambitious will work either. This formulation is essentially SurCo-zero, except that standard SurCo estimates the cost vector of a linear objective whereas ours parameterizes an MIQP.

Experiment 2: 64-env-random.

Sixty-four parallel environments with the target pose fixed but the initial T-block pose resampled uniformly at every training iteration. This forces the network to learn a contextual policy $g_\theta(y)$ that generalizes across starting configurations rather than memorizing a single action. All 64 environments share the same target so the policy’s job is purely to condition on the initial pose. This formulation does not correspond exactly to any of the SurCo variants but the closest match is SurCo-prior. The primary difference is that SurCo-prior assumes a fixed finite training set $\{y_i\}_{i=1}^N$, whereas we sample fresh contexts from a continuous distribution at every iteration. Second, as in our earlier setup, the predicted costs parameterize an MIQP rather than a linear program.

Experiment 3: effect of lambda.

We conducted three ablation studies measuring the *face commitment margin*—the gap between the lowest and second-lowest face logit cost, $c_f^{(2)} - c_f^{(1)}$ —as a proxy for how decisively the network commits to a single face. A high margin indicates that one face is substantially cheaper in the MIQP objective, making the Gurobi solution robust to small perturbations; a low margin indicates faces are nearly tied and the chosen face may flip under minor context changes.

Experiment 4: randomized-smoothing-gradient-exploration.

To investigate why the model fails to learn an effective contextual policy, we design an experiment that mirrors Experiment 1 but trains on four identical environments in parallel rather than only 1. Again, the target pose and spawn locations stay the same throughout training. The same target pose is used for every environment. For each environment we visualize the gradient of the task loss L with respect to the cost vector c . However, plotting the per-environment gradient averaged over all M Monte Carlo samples turns out to be insufficient as it hides the variance contributed by individual perturbed solves and obscures how different face flips affect the estimator. We therefore decompose the estimator on a per-face basis, asking: *what would the mean gradient look like if we restricted the Monte Carlo sum to only those samples whose*

perturbed solve selected face i ? To answer this, we partition the M draws by their resulting face choice and inspect the conditional mean contribution from each subset.

6 Results

Experiment 1: 1-env-fixed.

In the 1-env-fixed setting the system converges quickly: the training loss drops to near its final value within roughly ten iterations (Figure 3). Further, through visual inspection it is evident that this value is in the optimal region for this task (Figure 4). Because the context y is identical at every step, the network only has to discover a single high-quality action $(\mathbf{f}, s^*, \alpha^*)$ for that one configuration, and the randomized-smoothing estimator is evidently informative enough to identify it almost immediately. By iteration 14 the resulting rollout (Figure 5) shows the pusher contacting the T on the correct face and driving it cleanly toward the target, confirming both that the gradient pipeline is wired correctly and that the MIQP action parameterization is expressive enough to solve a single-configuration instance of the task.

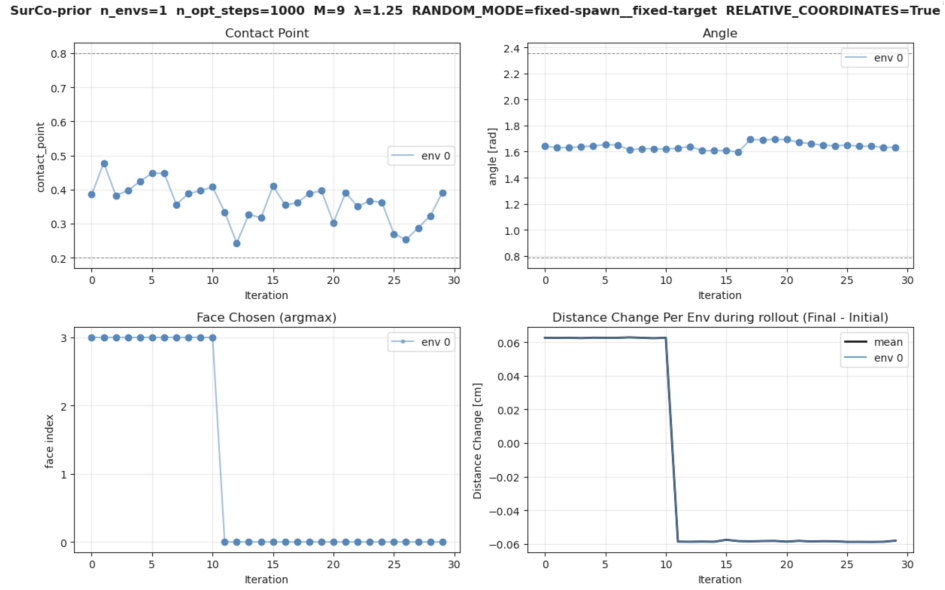


Figure 3: Convergence results for the 1-env-fixed experiment. An approximately optimal solution is found after roughly 10 gradient steps. The top two plots show the continuous action values (contact point s^* and angle α^*) over training iterations; the bottom-left plot shows the discrete face selection, and the bottom-right plot shows ΔL .

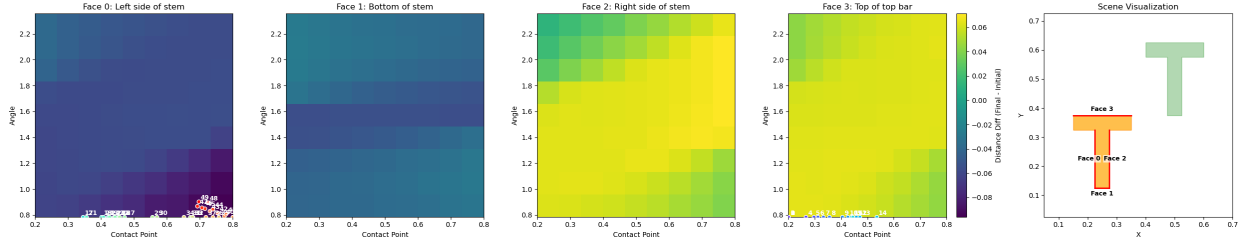


Figure 4: Task-loss landscape ΔL over the action space for the 1-env-fixed initial configuration. The four heatmaps correspond to the four faces, ordered 0, 1, 2, 3 from left to right; within each heatmap, the x -axis is the contact point s and the y -axis is the angle α . Lower values (darker) are better. The rightmost panel shows the initial T pose and the goal pose; the block trajectory is not drawn.

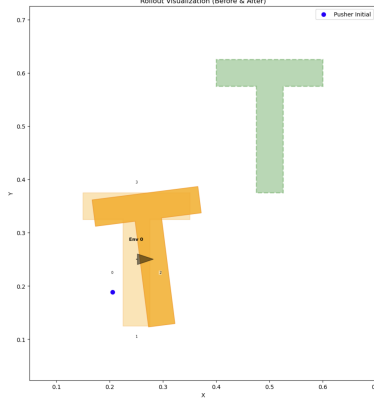


Figure 5: Rollout at training iteration 14 of the **1-env-fixed** experiment. The policy has learned to push the T in an approximately optimal direction.

Experiment 2: 64-env-random.

Despite extensive hyperparameter tuning — sweeping the learning rate, perturbation scale λ , number of Monte Carlo draws M , and the regularization coefficients — the system fails to learn an effective policy: the mean ΔL across environments hovers near zero throughout training (Figure 6), indicating that the learned actions are no better on average than randomly chosen ones. Performance degrades sharply as soon as more than two environments are trained on simultaneously, which suggests the gradient signal from the randomized-smoothing estimator is uninformative once the network must satisfy several distinct contexts at once. The nature of the gradients is explored in experiment 4.

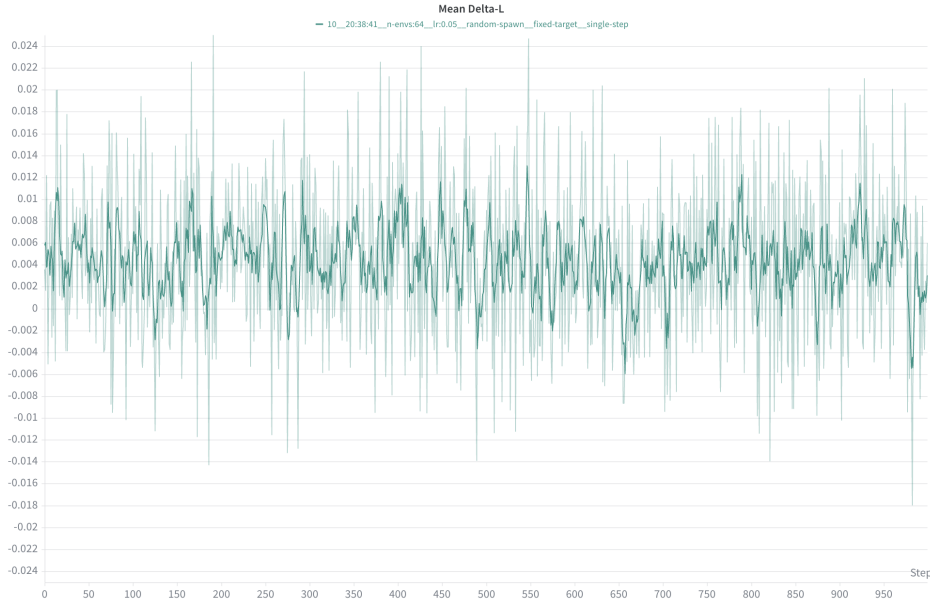


Figure 6: Mean task-loss change ΔL across environments over 1000 training iterations for the **64-env-random** experiment. The metric stays near zero, indicating that the learned actions perform no better than random. Longer training times (50000+ steps) does not result in better performance.

Experiment 3: effect of lambda.

Lambda sweep. We first swept $\lambda \in \{1.0, 1.25, 1.5, 1.75\}$ with all other hyperparameters held at their defaults ($\beta = 0.25$, $N = 16$ environments, 100 iterations). All four values produced commitment margins

that remained flat near 0.03–0.05 throughout training with no consistent monotone ordering across λ . This suggests that λ alone does not govern face commitment in this range, and that the training–evaluation misalignment described by Berthet et al. [2020] is not the primary driver of throttling.

Regularization sweep. We then held $\lambda = 1.25$ fixed and swept the face logit regularization coefficient $\beta \in \{0.0, 0.05, 0.25, 1.0\}$. The results show a clear separation: at $\beta = 0.25$ the margin is suppressed throughout training, while $\beta = 0.0$ and $\beta = 0.05$ allow the margin to grow to ~ 0.28 and ~ 0.22 respectively by iteration 100. At $\beta = 1.0$ the margin collapses toward zero. This indicates that the ℓ_2 penalty on face logits is strong enough to counteract the task gradient before any logit can develop meaningful separation—the network is actively prevented from committing to a face by its own regularization.

Lambda sweep under reduced regularization. Repeating the λ sweep at $\beta = 0.05$ raised the overall margin level to ~ 0.04 – 0.11 , confirming that reducing regularization improves commitment. However, the four λ values again showed no consistent ordering, suggesting that the λ misalignment effect is secondary to and largely masked by regularization in this hyperparameter range.

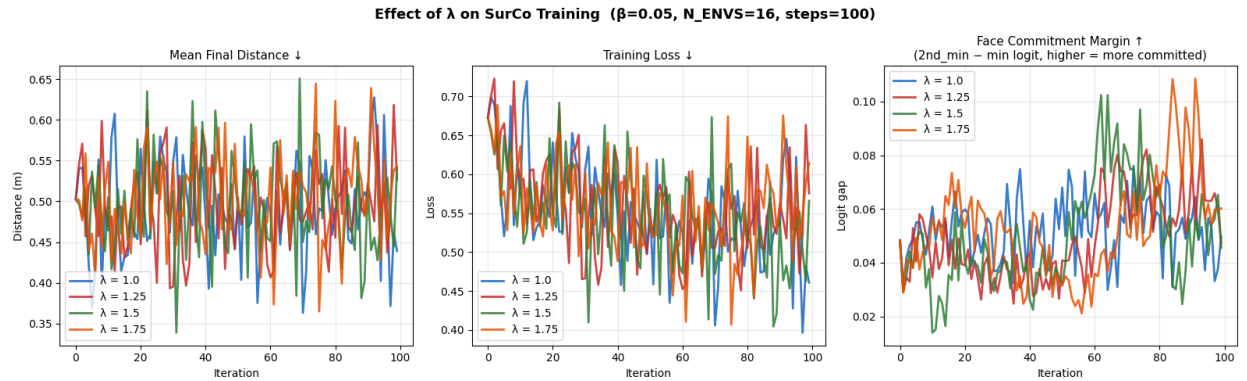


Figure 7: **Tuning Lambda** does not fundamentally fix the misalignment issue that we mention above, still having unpredictable behavior. Values were selected post regularization tuning.

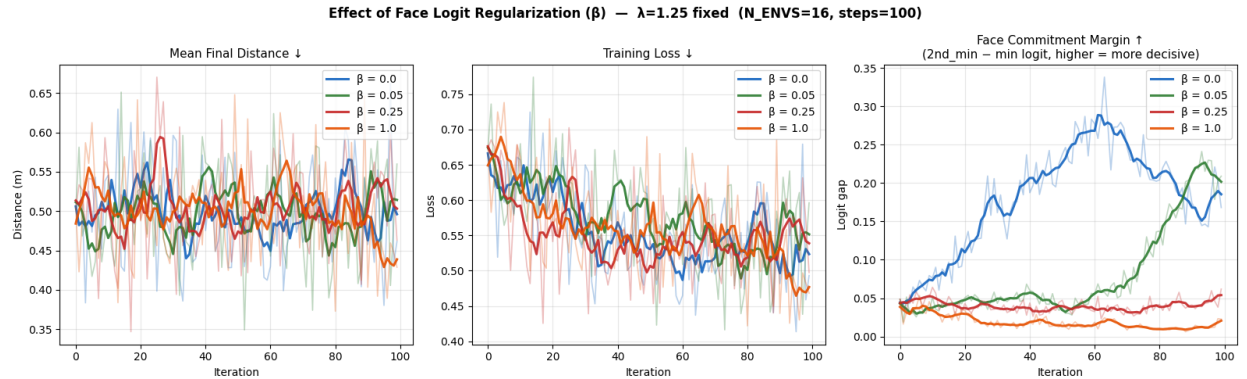


Figure 8: **Tuning Regularization magnitude.**

Experiment 4: randomized-smoothing-gradient-exploration.

Across training, we observe that environment 0 settles on a face choice that is strictly suboptimal: the selected face alternates between 2 and 3, both of which push the T away from the goal, while faces 0 and 1 would push it toward the target. The system never escapes this bad face, suggesting that the gradient signal at c is failing to point toward the correct optimum.

We hypothesize that this is a structural failure of the randomized-smoothing estimator caused by the argmin’s insensitivity to perturbations that do not flip the selected face. Recall that face selection is the argmin of the four face cost entries $c_0, \dots, c_3$. Consider a setting in which the current cost is $c = [1, 1, 1, 0]$, so face 3 is selected, and the true optimum is face 0. A perturbation $\delta = [-0.9, 0, 0, 0]$ — which moves c_0 toward the value that would correctly flip the choice to face 0 — still leaves face 3 as the argmin (since $0 < 1 - 0.9$). The perturbed rollout therefore evaluates the same action as the unperturbed one, the cost difference $L_k - L_0$ is near zero, and the Monte Carlo estimator assigns the direction δ a vanishing weight.

This effect is directly visible in Figure 9. While the dominant gradient on c_0, c_1, c_2 is negative — consistent with the network trying to reduce those entries to flip selection — there are persistent positive contributions as well. These positive gradients come from Monte Carlo samples whose face flips landed on suboptimal alternatives, and they pull c in exactly the opposite direction. The two signals partially cancel, and the resulting update is too weak and too noisy to escape the bad basin. Figures 10 and 11 confirm the behavioral consequence: after 250 iterations, environment 0 still selects face 2 and registers a positive ΔL , while the other environments in the batch have made progress.

Removing these “sharp edges” in the cost-to-action mapping is a natural direction for future work. One option is to replace the argmin face selection with a softer parameterization that distributes gradient signal across faces even when the discrete choice does not flip — for example, a temperature-controlled softmax over face logits or a differentiable top- k relaxation. Finding a parameterization that retains the expressiveness of the MIQP while admitting smoother gradients remains an open question.

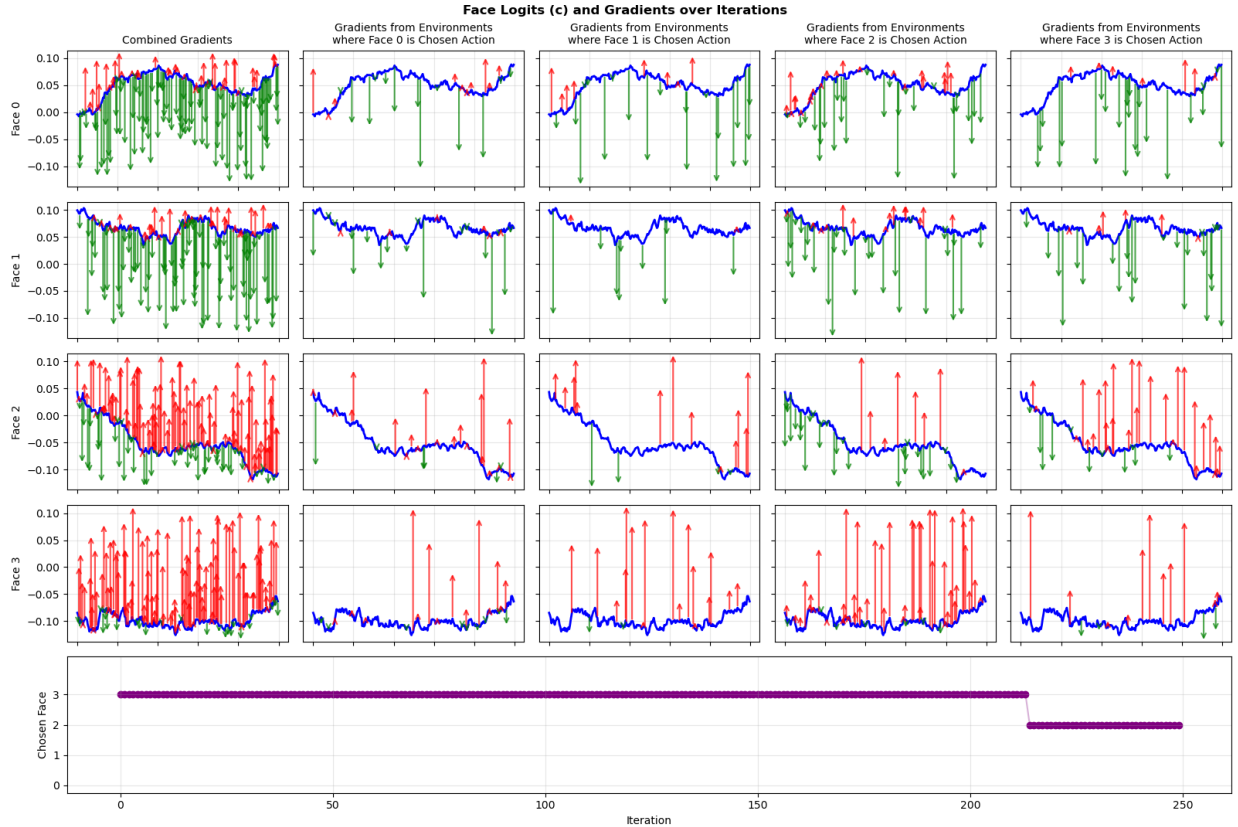


Figure 9: Gradient of the task loss with respect to c for environment 0 in the randomized-smoothing-gradient-exploration experiment, decomposed by which face the perturbed solve selected. Each subplot’s y -axis shows the value of c_i across training iterations; the leftmost column shows the current c and the direction of the summed Monte Carlo gradient at every step. The bottom plot shows the selected face per iteration: it remains at 3 before switching to 2. Both faces push the T away from the goal regardless of the continuous action values.

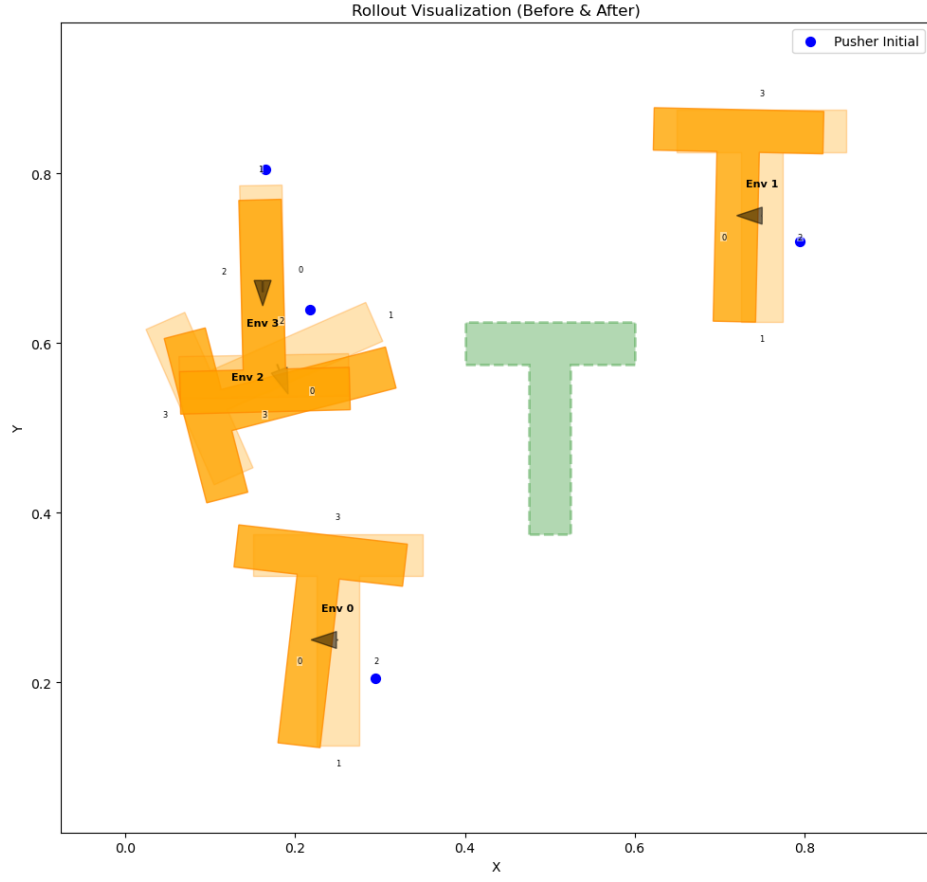


Figure 10: Rollout at training iteration 250 for the `randomized-smoothing-gradient-exploration` experiment. All four environments are shown overlaid in the same workspace (the Ts do not physically interact); environment 0 is the T at the bottom left. Its selected face is 2, which pushes the T directly away from the target. Selecting face 0 or 1 would be required to move toward the goal.

SurCo-prior n_envs=4 n_opt_steps=1000 M=9 $\lambda=1.25$ RANDOM_MODE=fixed-spawn_fixed-target RELATIVE_COORDINATES=True

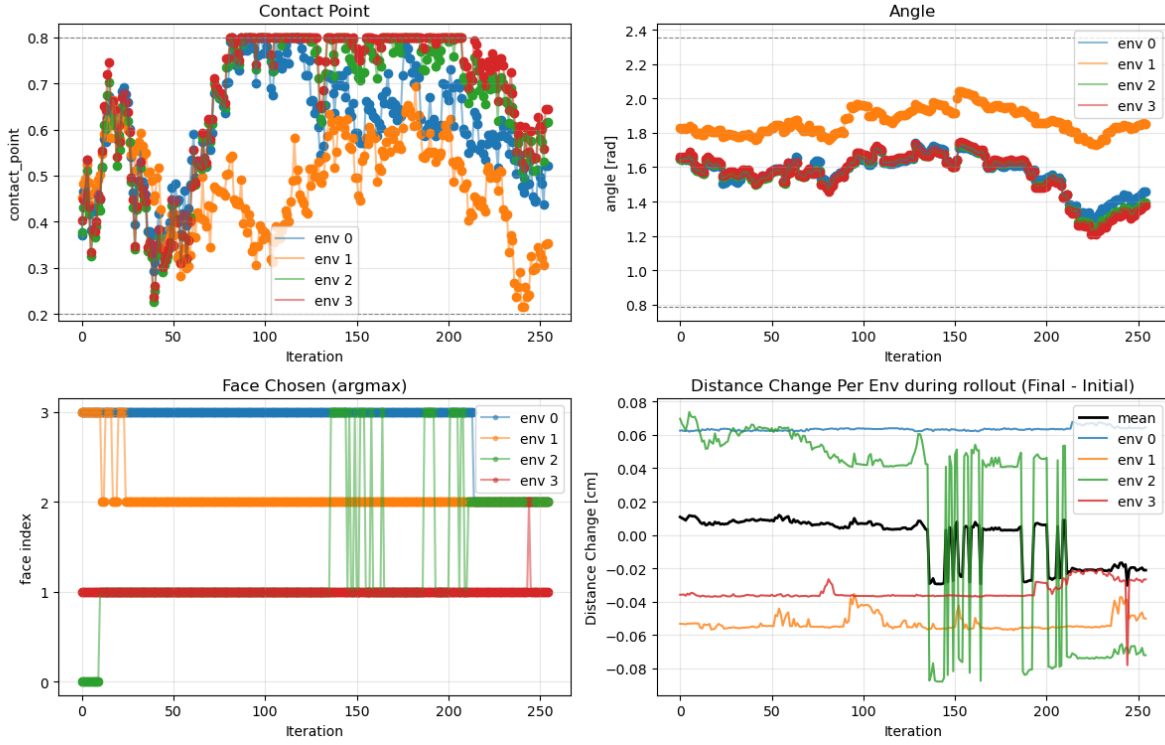


Figure 11: Per-environment training performance after 250 iterations for the randomized-smoothing-gradient-exploration experiment. Environment 0 has not converged on a useful face, as indicated by its positive ΔL (the pusher moves the T away from the goal).

Future work. The most important next step is to reformulate the problem so that it yields informative gradients, enabling the MLP to efficiently learn good actions. Once that is in place, extending the definition of an action to a sequence of pushes within a single solve would open the door to longer-horizon planning.

7 Feedback Incorporation

This section describes the feedback we received during the project presentation and code review, and how we incorporated it into the final report and codebase.

- **SurCo variant clarification.** Experiment 1 and 2 most closely align with SurCo-zero, SurCo-hybrid respectively, however neither of them are exact matches. This is explained in more detail in the Experiments section.
- **Single- vs. multi-environment nomenclature.** We have dropped the “single environment” / “multiple environment” framing in favor of named experiments (1-env-fixed and 64-env-random) whose learning problems are described explicitly. See the Experiments section for details.
- **Neural network structure and feature representation.** The network architecture is described in detail in the Approach section. Following the provided feedback, we now encode the task’s translation and rotational symmetries directly into the input: the context vector is expressed in the goal pose’s body frame, so any rigid transformation of the workspace that preserves the relative pose between the T and the goal leaves the input unchanged. The network is therefore not asked to learn SE(2) invariance from data — it is built into the feature representation.

- **Single-environment convergence behavior.** Training for Experiment 1 was run for 250 iterations, though the policy reaches an approximately optimal action within the first 10 (see Figure 3). For Experiment 2 we performed an extensive hyperparameter search over the network’s hidden size and depth, the learning rate, the choice of optimizer, the regularization coefficients, and the perturbation scale used in randomized smoothing. The settings reported for both experiments are the best configurations found in this search.
- **Diagnosis of poor numerical performance.** As argued in the Results section, we believe the dominant factor is uninformative gradients arising from the structure of the formulation itself — in particular, the arg min-induced sharp edges in the cost-to-action mapping examined in Experiment 4. The other candidate explanations are less compelling. Rotational symmetry is now baked into the input by expressing the context in the goal frame, so the only remaining rotational quantity the network must reason about is the scalar orientation error $\Delta\theta$, which an MLP should be capable of handling. SGD was tuned extensively, and individual training runs for Experiment 2 were extended up to 20 hours on a P100 GPU without reaching a useful solution, ruling out an under-training or learning-rate explanation. The vertex-solution issue raised in the feedback was addressed by switching from a MILP to an MIQP surrogate, which permits interior-point solutions on the continuous variables.
- **Extreme-point behavior.** The optimal action for Experiment 1 does in fact lie in the interior of the feasible region rather than at a vertex, as visible in the action-space cost landscape in Figure 4 — the low-cost basin for the correct face is centered well away from the corners of the (s, α) box. Switching from a MILP to an MIQP surrogate addresses this directly: the quadratic terms on s and α let the solver return solutions in the interior, so the learned policy is no longer confined to vertex actions.
- **Randomized smoothing as a remedy.** Considerable effort went into tuning the perturbation scale λ . The trade-off is fairly sharp: too small a λ rarely flips the face selected by the solver, so the Monte Carlo samples mostly produce zero cost differences and the estimator carries little information; too large a λ drives the perturbed costs far from c , which both destabilizes training through large gradient magnitudes and prevents the optimizer from refining the continuous targets within a fixed face. The value $\lambda = 1.25$ reported in the Approach section provided a good tradeoff.
- **Multi-step MPC results.** Multi-step MPC requires a policy that produces good actions across the full distribution of T poses, not just at a single configuration. Because Experiment 2 did not yield such a policy, executing it in closed loop would simply move the T approximately at random, and the resulting trajectories would not be informative. We therefore omit those results rather than report them as evidence of multi-step performance. The architecture does exist however.

8 Conclusion

We presented a demonstration-free learning pipeline for planar push manipulation that combines a learned cost predictor, a Gurobi MIQP solver, and a differentiable physics simulator. The key technical contribution is the end-to-end training signal obtained by composing JAX’s automatic differentiation with a custom randomized-smoothing VJP that propagates gradients through the combinatorial face-selection step. Experiments show the method can effectively learn actions for a single, fixed environment, but struggles when in a randomized environment.

References

- Brandon Amos and J. Zico Kolter. OptNet: Differentiable optimization as a layer in neural networks. In *Proceedings of the 34th International Conference on Machine Learning*. PMLR, 2017.
- Quentin Berthet, Mathieu Blondel, Olivier Teboul, Marco Cuturi, Jean-Philippe Vert, and Francis Bach. Learning with differentiable perturbed optimizers. In *Advances in Neural Information Processing Systems*, volume 33, pages 9508–9519, 2020.

- James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018. URL <http://github.com/jax-ml/jax>.
- Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems*, 2023.
- Filipe de Avila Belbute-Peres, Kevin Smith, Kelsey Allen, Josh Tenenbaum, and J Zico Kolter. End-to-end differentiable physics for learning and control. *Advances in Neural Information Processing Systems*, 31, 2018.
- Adam N. Elmachtoub and Paul Grigas. Smart “predict, then optimize”. *Management Science*, 68(1):9–26, 2022.
- Aaron Ferber, Taoan Huang, Daochen Zha, Martin Schubert, Benoit Steiner, Bistra Dilkina, and Yuandong Tian. Surco: Learning linear surrogates for combinatorial nonlinear optimization problems. In *Proceedings of the 40th International Conference on Machine Learning*, Proceedings of Machine Learning Research. PMLR, 2023.
- Diego Ferigo, Fabio Bergonti, Giuseppe L’Erario, Francesco Nori, and Daniele Pucci. JAXsim: A differentiable physics engine and multibody dynamics library for control and robot learning, 2023. URL <https://github.com/ami-iit/jaxsim>.
- Pete Florence, Corey Lynch, Andy Zeng, Oscar A Ramirez, Ayzaan Wahid, Laura Downs, Adrian Wong, Johnny Lee, Vincent Vanhoucke, and Jonathan Tompson. Implicit behavioral cloning. In *Conference on Robot Learning*. PMLR, 2022.
- Gurobi Optimization, LLC. Gurobi optimizer reference manual, 2023. URL <https://www.gurobi.com>.
- Kevin M. Lynch and Matthew T. Mason. Manipulation and active sensing by pushing using tactile feedback. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 1992.
- Matthew T. Mason. Mechanics and planning of manipulator pushing operations. *The International Journal of Robotics Research*, 5(3):53–71, 1986.
- Marin Vlastelica Pogančić, Anselm Paulus, Vit Musil, Georg Martius, and Michal Rolinek. Differentiation of blackbox combinatorial solvers. In *International Conference on Learning Representations*, 2020.

Author Contributions

Jeremy Morgan: Implemented the physics code, final training pipeline, experiments 1,2,4, and contributed to the paper. **Changhui Hou:** Implemented the differentiable face via soft max steps, early versions of MLP, tested training code, implemented early test cases, and data generation code for debugging. **Leland Ling:** Implemented the initial SurCo training loop and custom JAX VJP, designed ablation experiments involving randomized smoothing (tuning lambda and regularization terms), verifying throttling, wrote/formatted deliverables.